

Fourth Assignment: Exact Algorithms for Combinatorial Optimization

Instructions

- Answer conceptual questions clearly and precisely.
- Show all reasoning in computational parts.
- For algorithmic parts, provide:
 - A flowchart
 - Clear pseudocode

1 Exact Algorithms in Optimization

Conceptual Questions

1. What is an exact algorithm?
2. What is the difference between exact algorithms and heuristic or approximation algorithms?
3. What role do bounds and relaxations play in exact algorithms?

2 Complete Enumeration (Brute Force)

Conceptual Questions

1. What is complete enumeration?
2. What is the computational complexity of brute-force search for combinatorial problems?
3. Why is brute force rarely practical for large-scale problems?
4. In which situations can brute force still be useful?

Exercise

Consider the following 0/1 knapsack problem:

$$\max 8x_1 + 6x_2 + 5x_3$$

subject to

$$4x_1 + 3x_2 + 2x_3 \leq 5$$

$$x_i \in \{0, 1\}$$

1. List all possible feasible solutions.
2. Compute the objective value for each solution.
3. Identify the optimal solution.
4. How many solutions must be checked in general for; i. a knapsack problem with n items, ii. a Traveling sales person problem with n cities, iii. the following IP?

$$\min 4x_1 + 6x_2 + 2x_3 + x_4 + 3x_5$$

$$\text{s.t. } x_1 + 2x_2 + x_3 \geq 4$$

$$3x_1 + x_4 + x_5 \leq 25$$

$$x_2 + x_3 + 2x_5 \geq 8$$

$$x_3 + x_4 \leq 20$$

$$x_1, x_2 \in \{0, 1\}$$

$$x_3 \in \{1, \dots, 10\}$$

$$x_4, x_5 \in \{j \in \mathbb{Z} \mid \text{mod}(j, 2) = 1, 0 < j < 20\}$$

3 Dynamic Programming

Conceptual Questions

1. What is the main idea behind dynamic programming (DP)?
2. What are overlapping subproblems?
3. What is the principle of optimality?
4. What types of optimization problems are suitable for dynamic programming?
5. What are the components of a dynamic program? Explain every component in details.

Exercise

Consider the following knapsack problem:

Profits: $p = (6, 4, 5)$

Weights: $w = (3, 2, 4)$

Capacity: $W = 5$

1. Write the dynamic programming recurrence relation.
2. Define every component of the corresponding DP.
3. Construct the dynamic programming table.
4. Determine the optimal solution.
5. What is the time complexity of the dynamic programming algorithm for knapsack?
6. What is forward dynamic programming?
7. Define every component of forward DP.
8. Compare the two DPs in all their aspects.

4 Branch-and-Bound

Conceptual Questions

1. What is the main idea of the Branch-and-Bound algorithm? Explain the components and steps in details.
2. What are the three fundamental components of the method?
3. What does it mean to prune a node?
4. Why can Branch-and-Bound dramatically reduce the search space?

Exercise

Solve the following integer program using a Branch-and-Bound tree:

$$\max 4x_1 + 3x_2$$

subject to

$$2x_1 + x_2 \leq 7$$

$$x_1 + 2x_2 \leq 7$$

$$x_1, x_2 \in \mathbb{Z}_{\geq 0}$$

1. Solve the LP relaxation.
2. Draw the Branch-and-Bound search tree.
3. Compute bounds at each node.
4. Indicate which nodes are pruned.
5. Identify the optimal integer solution.

5 Constraint Programming

Conceptual Questions

1. What is Constraint Programming?
2. How does Constraint Programming differ from Mixed Integer Programming?
3. What is a constraint satisfaction problem (CSP)?
4. What is a global constraint? Give some examples and explain each example in details.
5. What is a symmetry breaking constraint? Provide at least three examples.
6. What is constraint propagation?
7. What is domain reduction?
8. What is backtracking search?
9. In what types of problems is Constraint Programming particularly effective?

Exercise

Three tasks A , B , and C must be scheduled on a single machine.

Each task takes one unit of time and must be assigned to one of the time slots:

1, 2, 3

The following constraints must hold:

- Task A must be scheduled before task B .
- Task C cannot be scheduled at time slot 1.
- No two tasks may occupy the same time slot.

1. Define the variables and their domains.
2. Formulate the constraints.

3. Apply logical reasoning (constraint propagation) to reduce domains.
4. Determine all feasible schedules.
5. Explain how a backtracking search algorithm would solve this problem.

6 Decomposition Methods

Conceptual Questions

1. What is the general idea of decomposition methods in optimization?
2. What structural properties of optimization problems make them suitable for decomposition techniques?
3. What is Benders decomposition? Explain the conceptual idea, give the formal formulation, and describe the algorithm step by step. What are the underlying principles?
4. What is Dantzig–Wolfe decomposition? Provide the formal formulation and explain the algorithm and its components in detail. What are the underlying principles?
5. What are the main differences between Benders decomposition and Dantzig–Wolfe decomposition?
6. Why are decomposition methods important for solving large-scale optimization problems?

7 Cutting Planes and Branch-and-Cut

Conceptual Questions

1. What is a cutting plane? How can a cutting plane be generated? Explain every approach for generating such planes.
2. What is a valid inequality?
3. What is Branch-and-cut algorithm? Explain the steps and components in details.
4. How do cutting planes improve LP relaxations?
5. What is the difference between Branch-and-Bound and Branch-and-Cut?
6. Why are cutting planes important in solving large MILPs?

8 Column Generation and Branch-and-Price

Conceptual Questions

1. What is column generation? Explain the algorithm and components in details.
2. What is the pricing problem?
3. What is the difference between column generation and branch-and-price?
4. In which types of problems is column generation commonly used?

9 Algorithm Design

Flowchart and Pseudocode

Provide a flowchart and structured pseudocode for the following algorithms:

1. A Dynamic Programming algorithm for the knapsack problem.
2. A generic Branch-and-Bound algorithm for maximization problems.

Your flowcharts should clearly include:

- Initialization
- State expansion or branching
- Bounding or evaluation step
- Selection of the next node or state
- Termination condition